

Process Generator (process_generator.c) →Yahia+Badr

- Implement reading and parsing of the input file (ignore comment lines).
- Store process data (arrival time, runtime, priority, dependency, etc.).
- Ask user to choose scheduling algorithm and set parameters (e.g., quantum).
- Initialize scheduler and clock.
- send each process's data to scheduler at its arrival time. (message queue)
- Send signal to scheduler when all processes are sent.
- Clean up and release all IPC resources before exit.

Process (process.c) →Yahia+Badr

- Simulate a CPU-bound process.
- Receive runtime and other parameters from the scheduler.
- Run for the specified time using sleep or busy wait (as per design).
- Notify scheduler upon completion (via IPC).
- Handle termination and cleanup properly.

Scheduler (scheduler.c) →Magdy+Khaled

- Create data structures:
 - Process Control Block (PCB)
 - Ready queue
 - Data structures for waiting and finished processes
- Implement state transitions (Ready → Running → Blocked/Finished).
- Implement process forking, stopping, and resuming logic.
- Implement the **three scheduling algorithms**:
 - **Highest Priority First (Adaptive Priority)**
 - **Shortest Remaining Time Next (SRTN)**

- **Round Robin (RR)**
- Handle tie-breaking between processes.
- Manage arrival of new processes and context switching.
- Handle dependencies & Priority Inversion
- Update process statistics: waiting time, turnaround time, weighted turnaround time.
- Compute performance metrics:
 - CPU utilization
 - Average waiting time
 - Average weighted turnaround time
 - Standard deviation of WTA
- Write to scheduler.log (detailed events).
- Write to scheduler.perf (final performance summary).
- Handle process termination notifications.
- Free and clean up IPC resources at exit.
- Write brief explanations for each scheduling algorithm.

General:

- Document all used data structures and algorithms.
- Include assumptions or design choices made.